



Cairo University
Faculty of Computing and Artificial Intelligence
Department of Computer Science



جَامِعَةُ الْقَاهِرَةِ
CAIRO UNIVERSITY

Game Forge

Supervised by
Dr. Samar
Hesham
TA. Nada Nasser

Implemented by

20220021	Ahmed Saber Ahmed
20220232	Omar Mohamed AbdElmelek
20220450	Amr Mohamed Sheriey
20220229	Omar Mahfouz Mohamed

Graduation Project
Academic Year
2025/2026
Midyear Short
Documentation

1. Abstract

Game development is a multidisciplinary and resource-intensive process that requires the integration of programming, game engine expertise, visual and audio asset creation, game design, and continuous iteration. This complexity poses a significant challenge for solo developers, students, and small teams, often preventing promising game ideas from progressing beyond early prototypes. Although recent advances in large language models and generative technologies have enabled partial automation of game design, code generation, and asset creation, existing solutions typically address these components in isolation and lack a unified, end-to-end workflow.

This project presents **Game Forge**, an MCP-based automated game generation server that transforms natural-language prompts into playable game prototypes. The system is designed around a Node.js backend that acts as a Model Context Protocol (MCP) client, coordinating a centralized MCP server composed of multiple adapters responsible for game planning, asset preparation, code generation, and build execution. Using the Godot engine, the system generates editable game projects and exports playable HTML5 builds that can be accessed directly through the browser.

The initial scope of Game Forge focuses on generating simple 2D top-down games with basic mechanics and win/lose conditions. The proposed system aims to lower the barrier to entry for game development by providing an integrated, automated pipeline that converts high-level ideas into functional game prototypes.

2. Background

Game development is a complex software engineering domain that integrates programming, game engines, asset production, interaction design, and testing, often requiring significant time, expertise, and collaboration. This complexity poses a challenge for individual developers, students, and small teams with limited resources. Recent advances in artificial intelligence, particularly large language models and generative systems, have introduced new possibilities for automating parts of the game development process, including content generation, code synthesis, and design assistance. However, existing AI-based tools typically address isolated stages of the pipeline, lacking an integrated workflow that unifies planning, implementation, and deployment into a coherent game prototype.

2.1 Motivation

While modern game engines such as Godot offer powerful capabilities for building cross-platform games, they remain largely inaccessible to individuals without programming or engine-specific expertise. Many creative individuals—such as storytellers, game designers, artists, and students—possess compelling game ideas and narratives but lack the technical skills required to transform these ideas into playable prototypes. As a result, their concepts often remain theoretical, limiting their ability to communicate, demonstrate, or validate their ideas.

The ability to generate a functional game prototype is essential for early-stage idea validation, demonstration, and academic or creative exploration. However, most existing tools either require significant technical expertise or address isolated tasks,² limiting their usefulness for non-technical users. This project aims to bridge the gap between creative ideation and technical implementation by enabling

the transformation of natural-language descriptions into playable game prototypes through coordinated multi-role large language models.

2.2 Beneficiaries

The proposed system targets a wide range of users, including:

- **Individual developers** who want to rapidly prototype game ideas without extensive setup or manual asset creation.
- **Students** learning game development concepts who need practical, playable examples generated from high-level descriptions.
- **Small development teams** with limited access to dedicated designers, artists, or sound engineers.
- **Game designers and hobbyists** who want to explore ideas without deep technical knowledge of game engines or scripting languages.

2.3 Main Application & Tech Stack

Game Forge is an AI-driven system that converts natural-language game ideas into coherent, playable game prototypes automatically. The proposed system is implemented as an integrated game generation platform built around multiple specialized LLM components. The user provides a natural-language description of a game concept, including gameplay mechanics, theme, and desired experience. The system processes this input through distinct LLM roles responsible for planning, code generation, theme and asset description, and audio concept generation.

Frontend: React + Vite + Tailwind CSS

Backend: Node.js + Express + MongoDB (Mongoose) + Socket.IO

Auth: JWT + bcryptjs

Game Engine: Godot 4.2.1 (GDScript, HTML5 export)

DevOps: Docker + Docker Compose + GitHub Container Registry

MCP Server: “Model Context Protocol”

3. Problem Definition

Transforming a high-level game idea into a playable prototype remains a manual and resource-intensive process that requires coordination across multiple development stages. Specifically:

1. **High barrier to entry:** Game development requires knowledge of gameplay logic, engine-specific scripting, asset pipelines, and build configuration. Individual developers, students, and small teams often lack the technical background needed to assemble these components into a functional prototype.
2. **Fragmented development workflow:** Core elements of a game—such as planning, code implementation, visual assets, audio assets, and execution—are typically developed and managed independently. Coordinating these components manually increases complexity and slows iteration cycles.
3. **Lack of orchestration and automation:** Existing solutions do not provide a standardized orchestration mechanism that ensures coherent communication between planning, asset

generation, code construction, and build execution stages. This fragmentation often results in incomplete or non-functional prototypes that fail to accurately represent the intended game design.

4. Related Work

Aspect	Existing Tools	Game Forge
Scope of automation	Most tools focus on a single aspect of game development, such as asset generation, music composition, or code assistance, without covering the full pipeline.	Provides an end-to-end pipeline that transforms natural-language descriptions into playable game prototypes.
Planning awareness	Existing systems typically lack explicit planning or rely on predefined templates and procedural rules.	Uses a dedicated planning LLM to reason about gameplay mechanics, structure, and constraints before generation.
Integration	Generated assets, code, and audio must be manually combined by the developer, often leading to inconsistencies.	Integrates planning, code generation, asset descriptions, and audio concepts through coordinated LLM roles.
Orchestration	No standardized orchestration mechanism exists between different generative tools.	Employs a Model Context Protocol (MCP) server to orchestrate communication and shared context across components.
Target users	Primarily designed for developers with technical expertise in engines and pipelines.	Designed for creative users, students, and non-programmers to prototype ideas without engine-specific knowledge.
Output usability	Outputs are often isolated artifacts (scripts, images, audio) that are not directly playable.	Produces coherent, playable game prototypes compatible with the Godot engine.
Extensibility	Tools are often tightly coupled to specific models or platforms.	Model-agnostic architecture supporting multiple LLM providers and roles.

5. Project Specifications

5.1 System Architecture

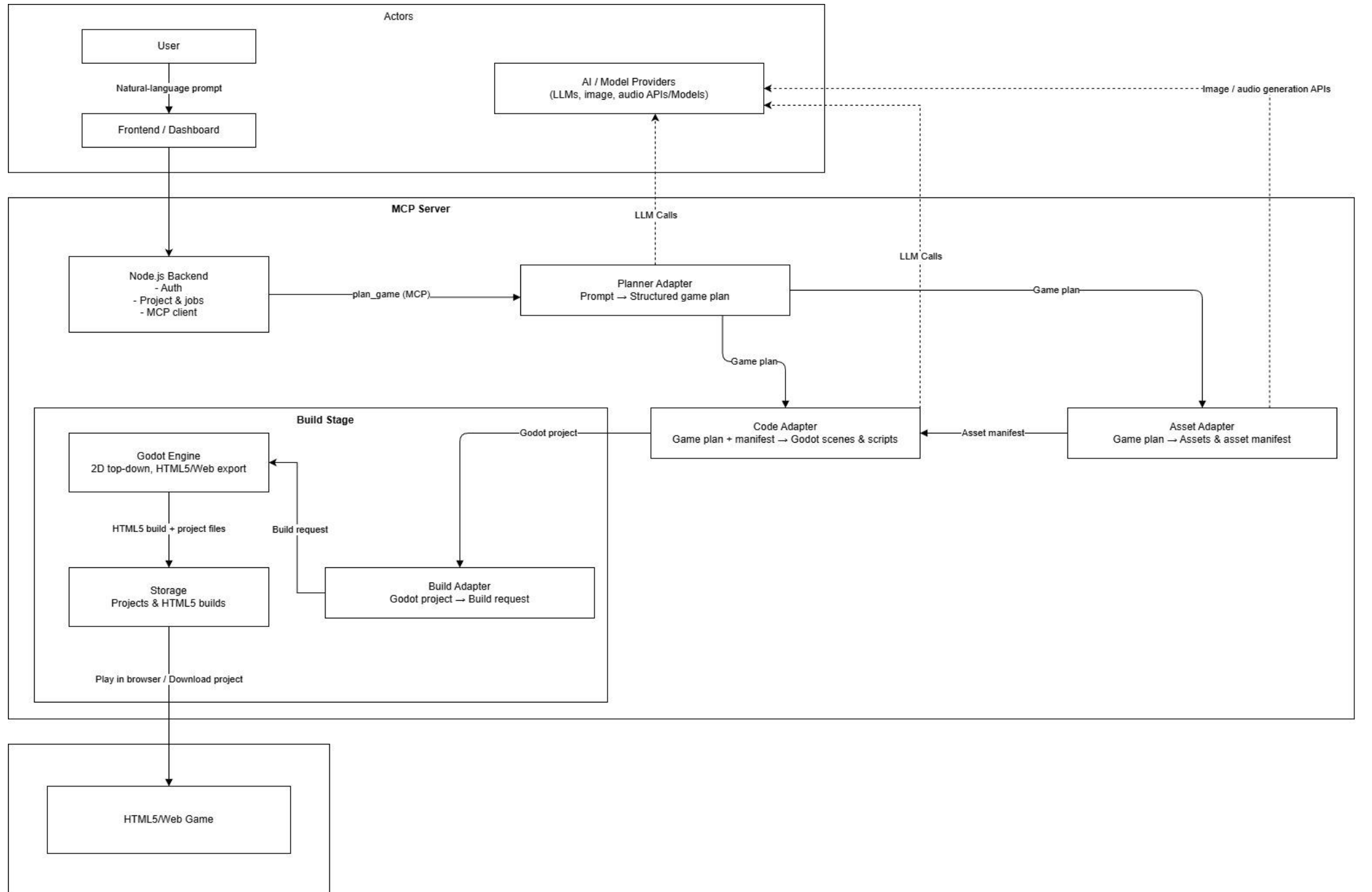
The Game Forge system adopts a modular client–server architecture that automates the generation of playable game prototypes from natural-language descriptions. The design emphasizes clear separation of concerns and end-to-end orchestration.

The architecture is organized into four main layers:

- **User Interface Layer**
A web-based frontend built with ReactJS allows users to submit game prompts, track generation progress, and access generated projects.
- **Backend Orchestration Layer**
A Node.js backend handles authentication, project management, and task coordination. It also acts as an MCP client responsible for initiating and managing generation workflows.
- **MCP Server Layer**
The Model Context Protocol (MCP) server coordinates the intelligent generation process through specialized internal adapters:
 - Planner Adapter: Converts user prompts into structured game design specifications.
 - Asset Adapter: Prepares or generates required visual and audio assets.
 - Code Adapter: Generates Godot-compatible scenes and scripts based on the game plan.
 - Build Adapter: Invokes the Godot engine to assemble and export the final game build.
- **Game Engine & Build Layer**
The Godot engine is used to assemble complete game projects and export playable HTML5 builds. Generated outputs are stored and made accessible to users for browser-based gameplay or download.

This layered architecture improves maintainability and scalability while enabling a fully automated pipeline from prompt submission to playable game output.

Figure 1. Block Diagram



5.2 Stakeholders

The Game Forge platform targets several real-world stakeholders who interact with or benefit from the system in different capacities.

Indie Game Developers:

Independent game developers represent the primary stakeholders of the system. They use the platform to rapidly transform game ideas into playable prototypes, enabling faster experimentation, iteration, and validation of concepts without requiring extensive development resources.

Students and Game Development Learners:

Students studying computer science, game development, or related disciplines use the platform as an educational tool to explore game design concepts, prototype projects, and understand the structure of game engines and development pipelines.

Academic Institutions and Supervisors:

Universities, instructors, and academic supervisors are indirect stakeholders who benefit from the system as a learning and evaluation tool for game development courses, student projects, and research activities.

Startup Teams and Small Studios:

Early-stage startups and small game studios may use the platform to prototype game ideas, pitch concepts, or develop early playable versions of games to support funding, collaboration, or community feedback.

Platform Administrators:

Platform administrators are responsible for managing the system, maintaining service availability, monitoring execution processes, and ensuring reliable operation for all users.

5.3 Functional Requirements

5.3.1 User Authentication and Account Management

- The system shall allow users to create an account using a valid email address and password.
- The system shall authenticate users before granting access to platform features.
- The system shall allow authenticated users to log out securely.

5.3.2 User Profile and Project Management

- The system shall allow users to view their profile information.
- The system shall allow users to view a list of their generated game projects.
- The system shall allow users to view detailed information about each project.
- The system shall allow users to manage their generated projects.

5.3.3 Prompt Submission and Validation

- The system shall allow users to submit natural-language descriptions of game ideas.
- The system shall create a unique project for each submitted game prompt.

5.3.4 Game Planning

- The system shall automatically convert user prompts into structured game design specifications.
- The system shall store generated game plans for future reference and traceability.

5.3.5 Asset Generation and Management

- The system shall generate visual assets required for the game.
- The system shall generate or prepare audio assets for the game.
- The system shall associate generated assets with their corresponding game projects.

5.3.6 Code and Scene Generation

- The system shall generate game scripts compatible with the Godot engine.
- The system shall generate game scenes and entities based on the game design plan.

5.3.7 Build and Execution

- The system shall assemble generated code and assets into a complete game project.
- The system shall export a playable HTML5 version of the generated game.
- The system shall track and manage game generation execution tasks.

5.3.8 Game Access and Distribution

- The system shall allow users to play the generated game through a web browser.
- The system shall allow users to download the generated Godot project files.

5.3.9 Status Reporting and Error Handling

- The system shall display execution status updates for game generation tasks.
- The system shall notify users if a game generation task fails or completes successfully.

5.4 Non-Functional Requirements

5.4.1 Security

- The system shall enforce secure authentication mechanisms for all registered users.
- The system shall ensure that user credentials are stored in an encrypted or hashed form.
- The system shall restrict access to administrative functions to authorized users only.

5.4.2 Performance

- The system shall respond to user authentication requests within an acceptable time under normal load.
- The system shall provide users with execution status updates for long-running game generation tasks.

5.4.3 Scalability

- The system shall support multiple concurrent users submitting game generation requests.
- The system shall allow future expansion of generation modules without major architectural changes.

5.4.4 Reliability

- The system shall handle failures during game generation gracefully without crashing the platform.
- The system shall notify users when a generation process fails or completes successfully.

5.4.5 Availability

- The system shall be available for use during normal operating hours except during scheduled maintenance.

5.4.6 Maintainability

- The system shall follow a modular design to allow independent updates of system components.
- The system shall support logging of system activities for debugging and maintenance purposes.

5.4.7 Usability

- The system shall provide a clear and intuitive user interface for submitting prompts and managing projects.
- The system shall present meaningful error messages that help users understand system behavior.

5.4.9 Portability

- The system shall allow generated projects to be exported in a format compatible with the Godot engine.

Figure 2. Use Case Diagram

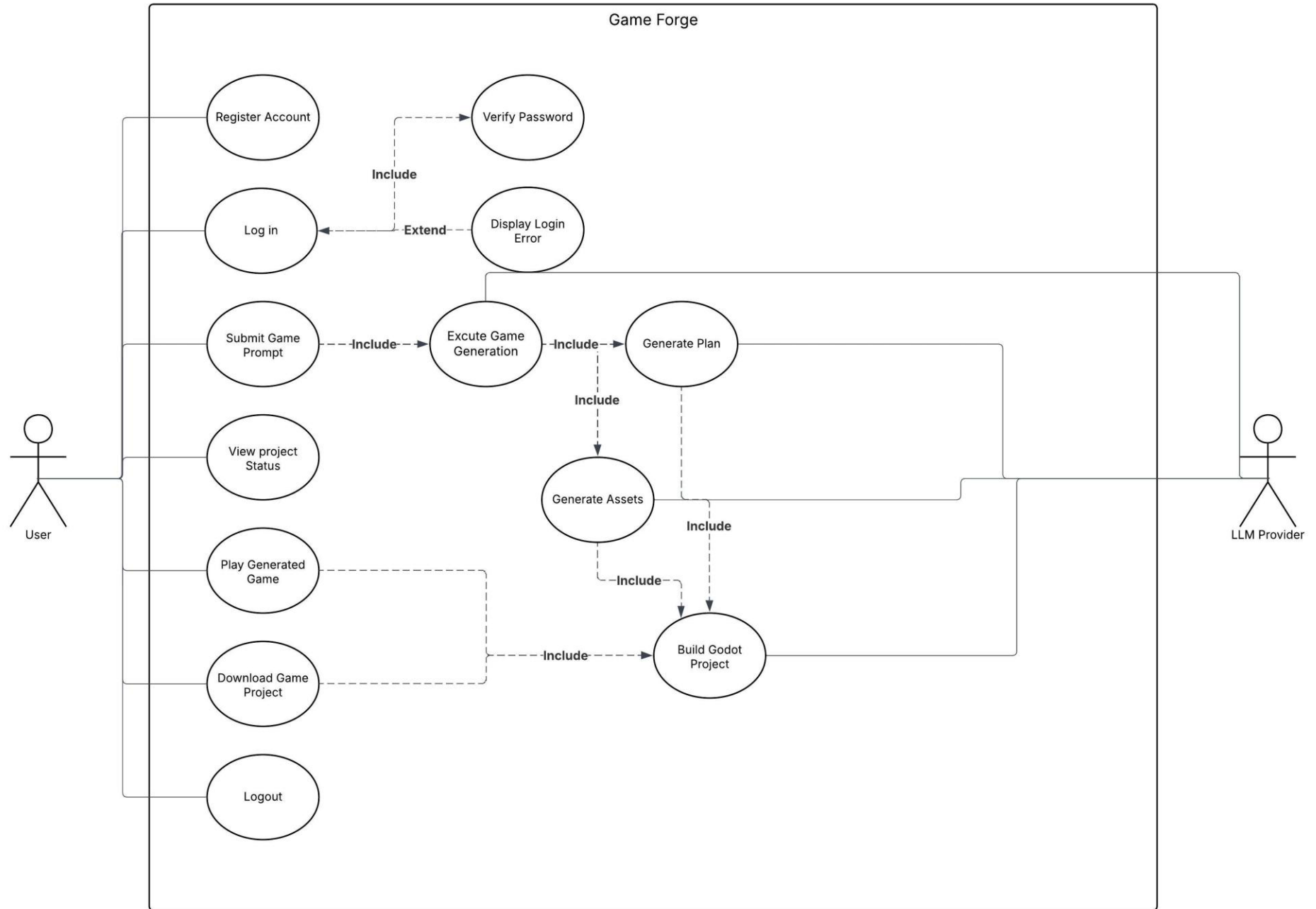


Figure 3. Class Diagram

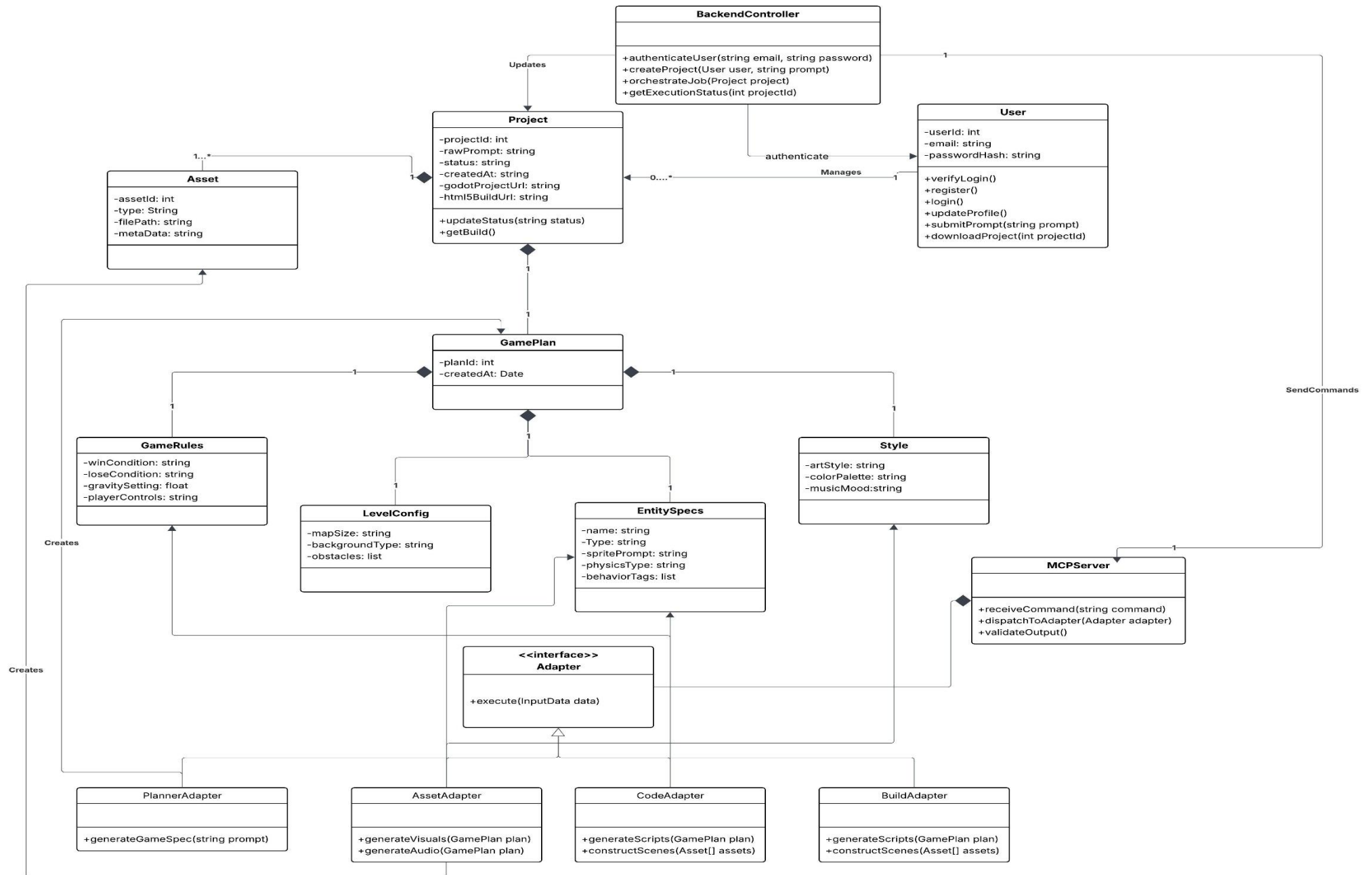


Figure 4. Sequence Diagram

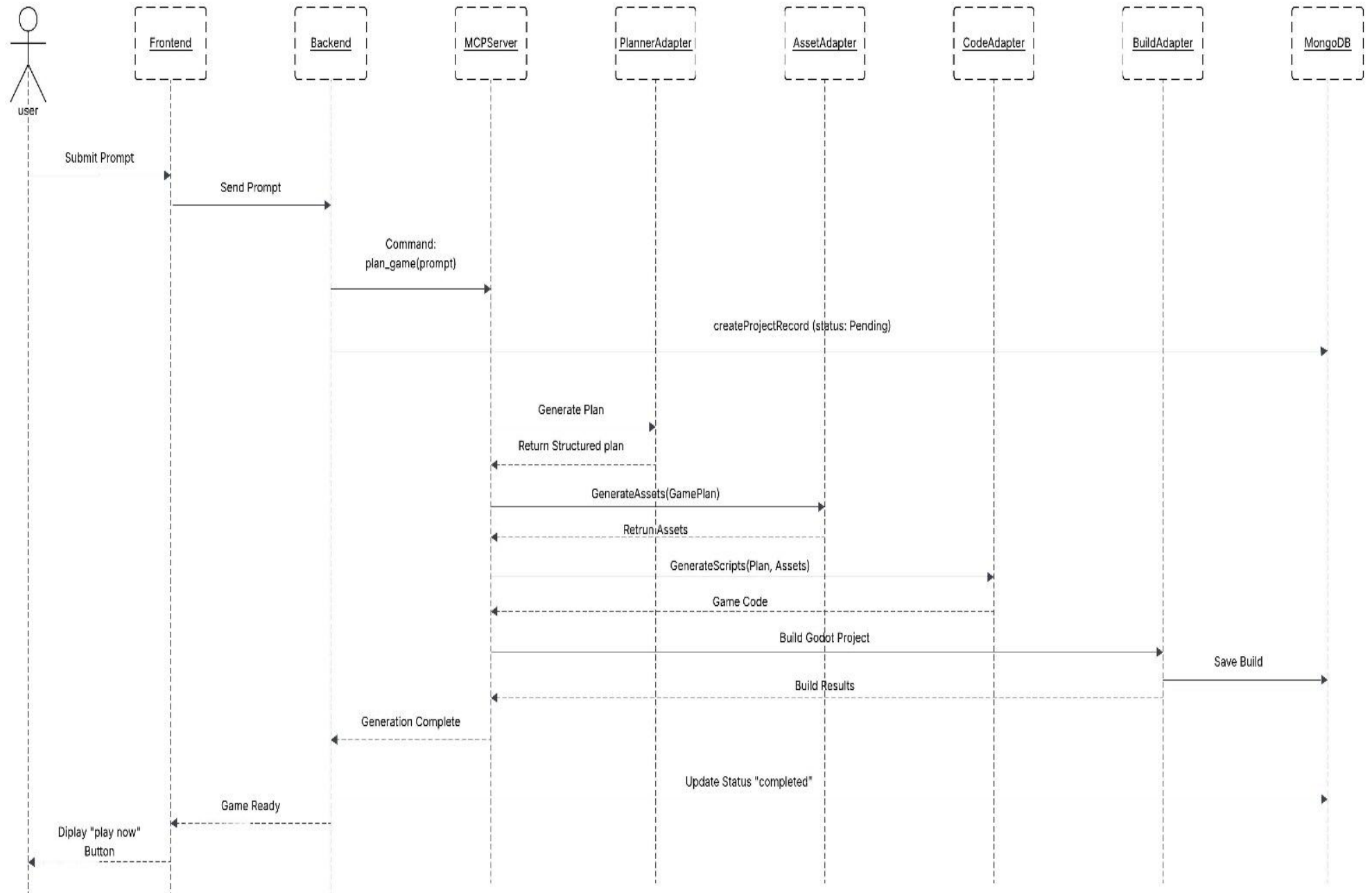
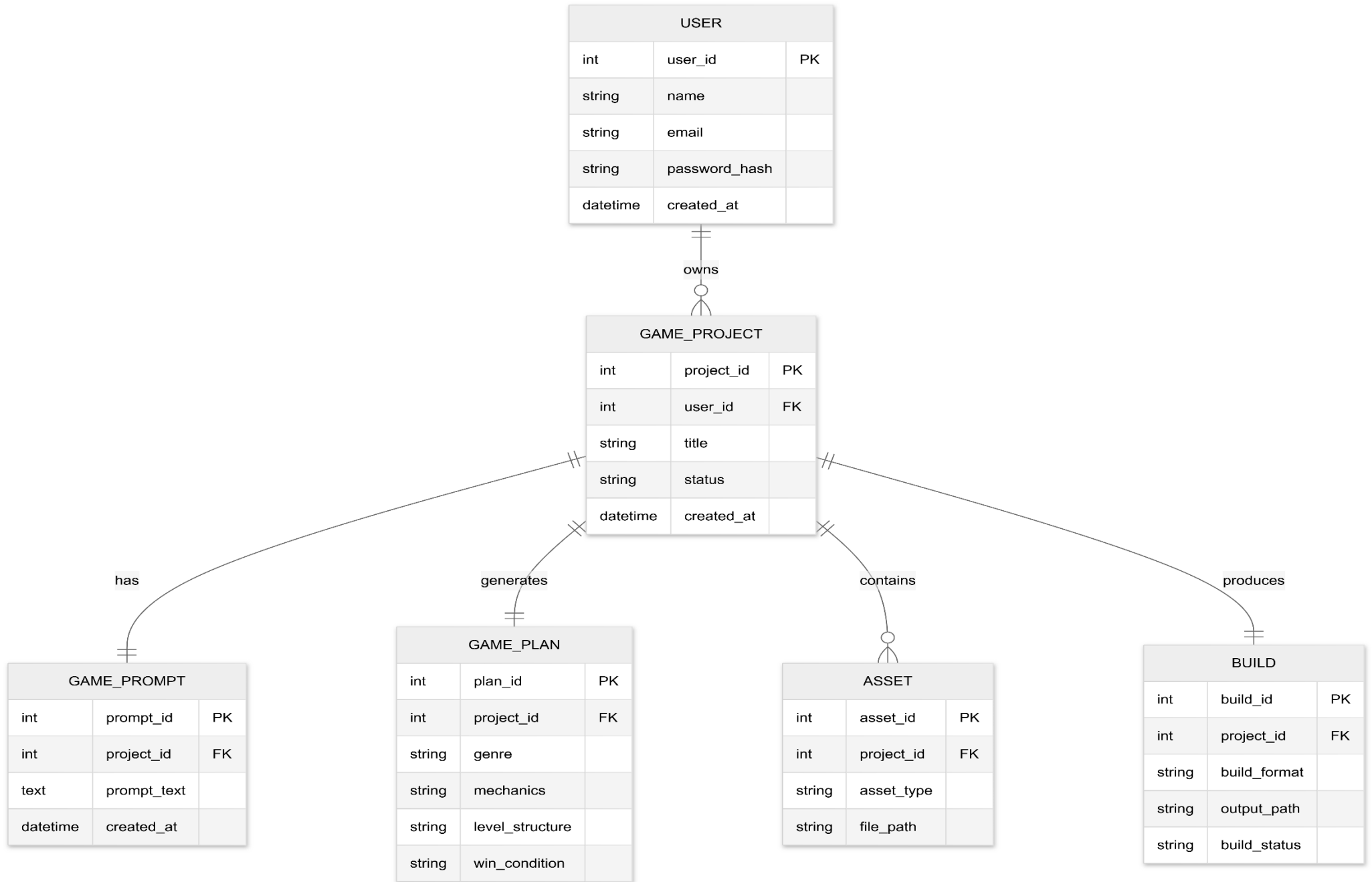


Figure 5. ERD



6. Work Plan

Task No.	Task Title	Description	Status
1	Project Idea & Literature Review	Identifying the problem domain, studying existing tools and research related to automated game generation, and defining the project scope.	Completed
2	Requirements Analysis	Analyzing functional and non-functional requirements, identifying stakeholders, and defining system objectives.	Completed
3	System Design	Designing the overall system architecture, block diagram, UML diagrams, and database schema.	Completed
4	Environment Setup	Setting up the development environment, version control repository, and project folder structure.	Completed
5	Frontend Development	Implementing the core frontend environment and initial design for the website covering most functionality	Completed
6	Backend Development	Implementing the core backend server responsible for user management, job orchestration, and communication with the MCP server.	Completed
7	Model Testing and configuration	Scripting python benchmarks for LLM accuracy and results comparison with multiple models and acquiring data and graphs	Completed
8	MCP Server & Adapters	Implementing the MCP server and its internal adapters for planning, code generation, and build execution.	March 15 th
9	Asset Generation Module	Integrating image and audio generation models and managing generated assets.	April 15 th
10	Game Assembly & Build	Programmatically assembling Godot projects, generating scenes and scripts, and exporting playable HTML5 builds.	April 25 th
11	System Integration	Integrating frontend, backend, MCP server, and game engine pipeline into a single automated workflow.	May 5 th
12	Testing & Optimization	Performing functional testing, handling edge cases, optimizing execution flow, and improving error handling.	May 15 th
13	Documentation & Presentation	Preparing the final project report, system documentation, diagrams, and defense presentation.	May 30 th